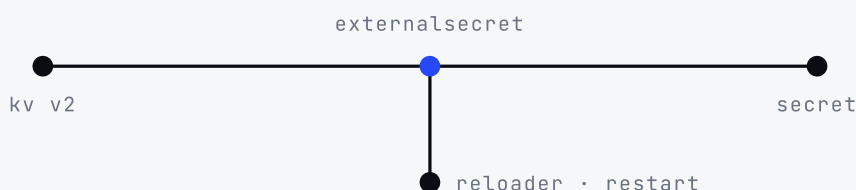


KUBERNETES SECRETS MIT ESO

Statische Secrets aus dem KV-Engine in den Cluster holen, rotieren, zurückschreiben – und warum ein rotiertes Secret noch keine rotierte Anwendung ist.



Kubernetes-Secrets mit ESO und OpenBao

Statische Secrets aus dem KV-Engine in den Cluster holen, rotieren, zurückschreiben – und warum ein rotiertes Secret noch keine rotierte Anwendung ist

Thomas Zachmann

17. September 2026

Inhalt

Worum es geht

- Warum von Hand, und warum ohne KI
- Ein Wort zu den Werten in diesem Leitfaden
- Lizenz und Haftung
- Für wen
- Die Bausteine
- Die Architektur in einem Bild

Teil I – KV v2 in OpenBao

- Der Mount
- Ein Secret anlegen
- Wer darf lesen: die Policy

Teil II – ExternalSecret: Feld für Feld

- SecretStore
- Das erste ExternalSecret
- Fehler 1: permission denied
- Das Deployment umstellen
- Nebenbei: die TTL-Falle aus N° 2, live

Teil III – Rotation

- Fehler 2: kv put ersetzt, es ergänzt nicht
- Fehler 3: Das Secret ist neu, der Pod nicht
- Fehler 4: Die Anwendung ist neu, das Passwort nicht
- Reloader: Restart bei Secret-Änderung

Teil IV – Die anderen Lesarten

- dataFrom.extract: das ganze Secret
- dataFrom.find: viele Secrets per Muster
- template: Dateien und zusammengesetzte Werte
- Lebenszyklus: creationPolicy und deletionPolicy

Teil V – PushSecret: die Gegenrichtung

- Wohin damit im Betrieb

Teil VI – Was schiefging, und warum

Teil VII – Betrieb

- Die Policy, wie sie am Ende aussieht
- Versionen als Sicherheitsnetz
- refreshInterval wählen
- Was in OpenTofu gehört
- Was noch offen ist

Anhang A – Alle Kommandos

Anhang B – Glossar

Worum es geht

Die meisten Secrets sind nicht dynamisch. Ein API-Key eines Drittanbieters, ein Admin-Passwort, ein Webhook-Token – Werte, die jemand einmal anlegt und die so lange gelten, bis jemand sie ändert. Sie gehören trotzdem nicht in ein Manifest, nicht in Helm-Values und nicht in eine CI-Variable. Sie gehören an einen Ort, der Versionen kennt, Zugriffe protokolliert und weiß, wer lesen darf: das KV-Engine von OpenBao.

Dieser Leitfaden zeigt, wie der **External Secrets Operator** (ESO) solche Werte in den Cluster holt und dort als ganz normales Kind: Secret ablegt – Feld für Feld, als ganzes Secret, per Muster über viele Secrets, oder als gerenderte Datei. Wie man rotiert, und was dabei alles *nicht* passiert. Und wie man in die Gegenrichtung schreibt, wenn ein Secret im Cluster entsteht und in OpenBao gesichert werden soll.

Es ist der vierte Teil einer Reihe. N° 1 baut den OpenBao im Cluster, N° 2 lässt eine Anwendung dynamische Datenbank-Credentials daraus beziehen, N° 3 beschreibt den OpenBao auf einer VM. N° 2 hat den statischen Fall übersprungen und das Miniflux-Admin-Passwort als offene Stelle im Manifest hinterlassen – hier wird sie geschlossen.

Alles wurde tatsächlich am Cluster durchgeführt. Die vier Fehler, die dabei auftraten, sind der Kern des Leitfadens.

Warum von Hand, und warum ohne KI

ExternalSecret ist ein Objekt mit zwanzig Feldern, und eine KI generiert es in drei Sekunden. Was sie nicht generiert, ist das Verständnis dafür, warum ESO aus `key: demo/miniflux` den Pfad `secret/data/demo/miniflux` macht, warum die Policy für `find` etwas anderes braucht als für `extract`, warum `kv put` ein Feld löscht, das man nicht angegeben hat, und warum ein Pod nach der Rotation noch das alte Passwort hat.

Der Maßstab: Wer diesen Leitfaden durchgearbeitet hat, kann für jedes Feld eines ExternalSecret sagen, welchen OpenBao-Pfad ESO daraus ableitet und welche Capability die Policy dafür braucht. Und er kann erklären, an welchen drei Stellen eine Rotation scheitern kann, obwohl OpenBao den neuen Wert längst hat.

Werkzeuge wie [nyrvex](#), das die Konfiguration von Secret Store und Identity Provider einer AI-Plattform generiert, nehmen einem diese Schritte später ab. Man sollte sie einmal selbst gegangen sein, um beurteilen zu können, was generiert wurde.

Ein Wort zu den Werten in diesem Leitfaden

Alles stammt aus einer Entwicklungsumgebung. Die Passwörter (`admin123`, `admin456`) sind bewusst abgedruckt, damit man sie im Ablauf wiedererkennt. Interne Adressen sind Cluster-DNS-Namen, die außerhalb des Clusters nichts bedeuten.

Lizenz und Haftung

Dieser Leitfaden steht unter CC BY 4.0: Er darf kopiert, weitergegeben und bearbeitet werden, auch kommerziell, solange der Autor genannt wird. Er wird ohne Gewähr bereitgestellt. Alles darin wurde in einer Entwicklungsumgebung durchgeführt; wer es in einer anderen Umgebung nachvollzieht, tut das auf eigene Verantwortung.

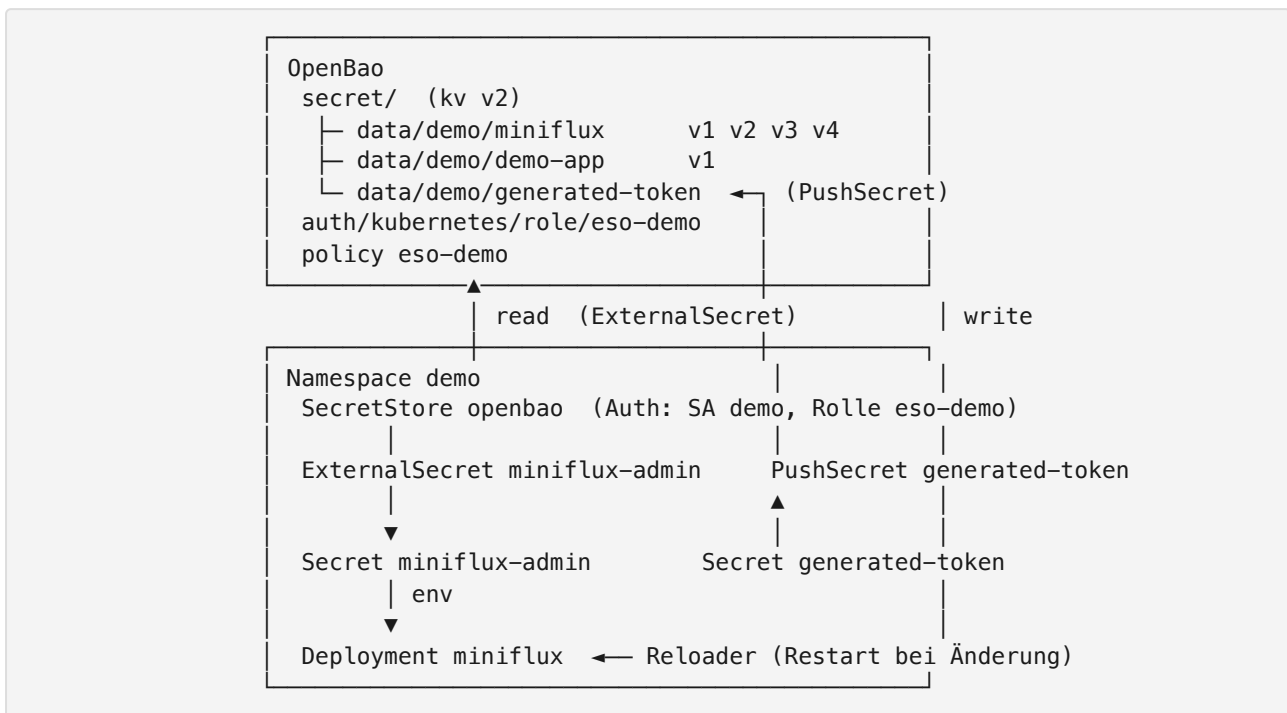
Für wen

Für Leserinnen und Leser, die N° 1 und N° 2 kennen oder die Grundlagen mitbringen: Kubernetes-Secrets, ServiceAccounts, Helm; in OpenBao die Kubernetes-Auth-Methode und das Konzept von Policies. Wer KV v2, Versionen und Policies von Grund auf lernen will, findet das in meinem Buch **Vault in Practice** (Leanpub, leanpub.com/vault-in-practice).

Die Bausteine

Baustein	Version	Rolle
OpenBao im Cluster (N° 1)	2.6.2	KV-v2-Engine unter secret/, Kubernetes-Auth mit Rolle eso- demo
External Secrets Operator	2.10.0	SecretStore, ExternalSecret, PushSecret
Stakater Reloader	Chart 2.2.17	startet Pods neu, wenn sich ein Secret ändert
Miniflux (N° 2)	2.3.3	die Anwendung, deren Admin- Passwort hier wandert

Die Architektur in einem Bild



Drei Dinge, die man aus dem Bild mitnehmen sollte:

1. **Der SecretStore ist die Verbindung, das ExternalSecret ist der Auftrag.** Der Store sagt *wohin* und *als wer*; das ExternalSecret sagt *was* und *in welcher Form*. Viele ExternalSecrets teilen sich einen Store.
2. **Jede Leserichtung braucht ihre eigene Capability.** `read` auf `data/...` für einzelne Secrets, `list` auf `metadata/...` für Muster, `create/update` auf `data/...` und `metadata/...` für Push. Die Policy wächst mit jedem Feature.
3. **ESO endet am Kubernetes-Secret.** Was der Pod damit macht – und ob er Änderungen bemerkt – ist nicht ESOs Problem. Dafür gibt es Reloader.

Teil I – KV v2 in OpenBao

Der Mount

```
/ $ bao read sys/mounts/secret -format=json | jq .data.options
{ "version": "2" }
```

KV **v2** ist versioniert: Jedes put erzeugt eine neue Version, alte bleiben lesbar, bis `max_versions` sie verdrängt oder jemand sie löscht. Für Rotation ist das wesentlich – man kann den vorherigen Wert nachlesen, wenn der neue nicht funktioniert.

Der Preis von v2 ist eine Eigenheit, die jeden Neuling einmal erwischt: Die API-Pfade heißen nicht `secret/demo/miniflux`, sondern `secret/**data**/demo/miniflux` für den Inhalt und `secret/**metadata**/demo/miniflux` für Versionen und Zeitstempel. Die CLI (`bao kv get secret/demo/miniflux`) versteckt das; Policies und ESO sehen die echten Pfade.

Ein Secret anlegen

```
/ $ bao kv put secret/demo/miniflux admin_username=admin admin_password=admin123
created_time      2026-09-17T10:31:03.905952914Z
version          1
```

Ein KV-Secret ist eine Map aus Feldern. `admin_username` und `admin_password` sind zwei Felder eines Secrets, nicht zwei Secrets. Das bestimmt später, wie ESO sie holt.

```
/ $ bao kv list secret/demo
Keys
----
demo-app
miniflux
```

Wer darf lesen: die Policy

ESO meldet sich mit dem ServiceAccount `demo` über die Kubernetes-Auth-Rolle `eso-demo` an und bekommt die gleichnamige Policy (N° 1, Teil III). Vor diesem Leitfaden sah sie so aus:

```
path "secret/data/demo/demo-app"      { capabilities = ["read"] }
path "secret/metadata/demo/demo-app"  { capabilities = ["read"] }
path "database/creds/demo-app"        { capabilities = ["read"] }
path "database/creds/miniflux"        { capabilities = ["read"] }
```

`secret/demo/miniflux` steht nicht drin. Das ist Absicht – der erste Fehler soll passieren, damit man ihn erkennt, wenn er später im Ernst passiert.

Teil II – ExternalSecret: Feld für Feld

SecretStore

Existiert seit N° 2, hier vollständig:

```
apiVersion: external-secrets.io/v1
kind: SecretStore
metadata:
  name: openbao
  namespace: demo
spec:
  provider:
    vault:
      server: http://openbao.openbao.svc:8200
      path: secret          # der Mount
      version: v2           # -> ESO fügt data/ und metadata/ selbst ein
      auth:
        kubernetes:
          mountPath: kubernetes
          role: eso-demo
          serviceAccountRef:
            name: demo
```

Ein SecretStore gilt für einen Namespace. Ein ClusterSecretStore gilt clusterweit und wird per secretStoreRef.kind: ClusterSecretStore referenziert – praktisch, wenn viele Namespaces denselben OpenBao nutzen, aber dann läuft jede Anfrage über *eine* Identität, und die Policy kann nicht mehr nach Namespace unterscheiden. Ein Store pro Namespace mit eigener Auth-Rolle ist die sauberere Wahl, sobald es mehr als einen Mandanten gibt.

Das erste ExternalSecret

Zwei Felder, zwei Keys im Kubernetes-Secret:

```

apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: miniflux-admin
  namespace: demo
spec:
  refreshInterval: 1h
  secretStoreRef:
    kind: SecretStore
    name: openbao
  target:
    name: miniflux-admin
    creationPolicy: Owner
  data:
    - secretKey: ADMIN_USERNAME
      remoteRef:
        key: demo/miniflux
        property: admin_username
    - secretKey: ADMIN_PASSWORD
      remoteRef:
        key: demo/miniflux
        property: admin_password

```

remoteRef.key ist der Pfad **ohne** Mount und **ohne** data/ – beides ergänzt ESO aus dem Store.
property wählt ein Feld. secretKey benennt den Key im Ziel-Secret; hier gleich so, wie Miniflux die Variablen erwartet.

Fehler 1: permission denied

```

$ kubectl apply -f k8s/miniflux-admin-external-secret.yaml
$ kubectl get externalsecret -n demo miniflux-admin
NAME                STATUS              READY   LAST SYNC
miniflux-admin      SecretSyncedError   False

```

Die status.conditions[].message sagt nur could not get secret data from provider. Die Ursache steht im Log des Operators:

```

$ kubectl logs -n external-secrets deploy/external-secrets | grep miniflux-admin
error processing spec.data[0] (key: demo/miniflux), err: cannot read secret
data from Vault: Error making API request.

URL: GET http://openbao.openbao.svc:8200/v1/secret/data/demo/miniflux
Code: 403. Errors:
  * permission denied

```

Zwei Dinge sind daran lehrreich. Erstens der Pfad: secret/data/demo/miniflux – aus key: demo/miniflux plus path: secret plus version: v2. Genau dieser Pfad muss in der Policy stehen. Zweitens der Ort der Meldung: Das ExternalSecret zeigt nur, *dass* es scheitert; *warum* steht beim Operator.

Die Policy um eine Zeile erweitern:

```

path "secret/data/demo/miniflux"      { capabilities = ["read"] }

```

Und statt eine Stunde auf den nächsten Sync zu warten, ihn erzwingen – jede Änderung an einer Annotation des ExternalSecret löst einen Reconcile aus:

```
$ kubectl annotate externalsecret -n demo miniflux-admin force-sync="$(date +%s)" --overwrite
$ kubectl get externalsecret -n demo miniflux-admin
NAME              STATUS      READY   LAST SYNC
miniflux-admin    SecretSynced  True    5s

$ kubectl get secret -n demo miniflux-admin -o jsonpath='{.type} {.data}'
Opaque {"ADMIN_PASSWORD":"...","ADMIN_USERNAME":"..."}

$ kubectl get secret -n demo miniflux-admin -o jsonpath='{.metadata.ownerReferences[0].kind}'
ExternalSecret
```

Das Secret gehört dem ExternalSecret (creationPolicy: Owner). Löscht man das ExternalSecret, nimmt die Garbage Collection das Secret mit. Dazu mehr in Teil IV.

Das Deployment umstellen

In N° 2 stand das Admin-Passwort im Klartext im Manifest. Jetzt:

```
- name: ADMIN_USERNAME
  valueFrom:
    secretKeyRef:
      name: miniflux-admin
      key: ADMIN_USERNAME
- name: ADMIN_PASSWORD
  valueFrom:
    secretKeyRef:
      name: miniflux-admin
      key: ADMIN_PASSWORD
```

```
$ kubectl apply -f k8s/miniflux-deployment.yaml
$ kubectl rollout status -n demo deploy/miniflux
deployment "miniflux" successfully rolled out
$ kubectl logs -n demo deploy/miniflux --tail=3
level=INFO msg="Running database migrations" current_version=132 latest_version=132
level=INFO msg="Skipping admin user creation because it already exists" username=admin
level=INFO msg="Starting HTTP server" listen_address=0.0.0.0:8080
```

Die zweite Zeile ist unscheinbar und wird in Teil III wichtig: Miniflux legt den Admin nur an, wenn er noch nicht existiert. ADMIN_PASSWORD wird danach **nicht wieder gelesen**.

Nebenbei: die TTL-Falle aus N° 2, live

Beim Rollout lief der alte Pod noch kurz weiter, und sein Log zeigte:

```
level=ERROR msg="Unable to fetch jobs from database" error="... pq: password
authentication failed for user \"v-kubernetes-miniflux-SkwRAKNo7i4xy0uFFD63-1789634099\""
```

Der dynamische Datenbank-User, mit dem der Pod vor zwei Stunden gestartet war, existierte nicht mehr – Lease abgelaufen, DROP ROLE ausgeführt. Der Pod hatte die neue DATABASE_URL nie gesehen, weil Umgebungsvariablen nur beim Start gelesen werden. N° 2 hatte das als offenes Risiko benannt; hier ist es eingetreten. Die Lösung kommt in Teil III und gilt für beide Secrets.

Teil III – Rotation

Fehler 2: kv put ersetzt, es ergänzt nicht

Der naheliegende Weg, ein Passwort zu ändern:

```
/ $ bao kv put secret/demo/miniflux admin_password=admin456
version          2
/ $ bao kv get -format=json secret/demo/miniflux | jq '.data.data | keys'
[ "admin_password" ]
```

admin_username ist weg. kv put schreibt eine **neue Version mit genau den angegebenen Feldern**. Version 1 hat noch beide, aber ESO liest die aktuelle:

```
$ kubectl get externalsecret -n demo miniflux-admin
NAME              STATUS              READY   LAST SYNC
miniflux-admin    SecretSyncedError   False   73s

$ kubectl logs -n external-secrets deploy/external-secrets | grep miniflux-admin
error processing spec.data[0] (key: demo/miniflux), err: cannot find secret
data for key: "admin_username"
```

Was ESO in diesem Zustand mit dem Kubernetes-Secret macht, ist die wichtigste Eigenschaft für den Betrieb:

```
$ kubectl get secret -n demo miniflux-admin -o jsonpath='{.data.ADMIN_PASSWORD}' | base64
-d
admin123
```

Nichts. Das Secret behält den letzten erfolgreich synchronisierten Stand. Ein Fehler in OpenBao oder in der Policy nimmt einer laufenden Anwendung nicht die Zugangsdaten weg – er verhindert nur, dass sie neue bekommt. Das ist richtig so, und man muss es wissen, weil ein False im READY deshalb nicht bedeutet, dass etwas kaputt *ist* – nur, dass es nicht mehr *aktualisiert* wird.

Der richtige Befehl ist kv patch:

```
/ $ bao kv patch secret/demo/miniflux admin_username=admin
version          3
/ $ bao kv get -format=json secret/demo/miniflux | jq '.data.data | keys'
[ "admin_password", "admin_username" ]
```

```
$ kubectl annotate externalsecret -n demo miniflux-admin force-sync="$(date +%s)" --overwrite
$ kubectl get externalsecret -n demo miniflux-admin
NAME              STATUS              READY   LAST SYNC
miniflux-admin    SecretSynced        True    5s
$ kubectl get secret -n demo miniflux-admin -o jsonpath='{.data.ADMIN_PASSWORD}' | base64
-d
admin456
```

Fehler 3: Das Secret ist neu, der Pod nicht

```
$ kubectl get pod -n demo -l app=miniflux -o custom-columns=NAME:.metadata.name,CREATED:.metadata.creationTimestamp
NAME                                CREATED
miniflux-86c6489dc4-6p2j5          2026-09-17T10:31:57Z

$ kubectl exec -n demo deploy/miniflux -- sh -c 'echo $ADMIN_PASSWORD'
admin123
```

Das Kubernetes-Secret sagt admin456, der Container sagt admin123. Ein Container liest env beim Start, und nichts hat ihn neu gestartet. Kubernetes aktualisiert gemountete Secrets im laufenden Container (mit Verzögerung), Umgebungsvariablen aber nie.

Fehler 4: Die Anwendung ist neu, das Passwort nicht

Angenommen, der Pod wäre neu gestartet und hätte admin456 in der Umgebung. Funktioniert der Login dann?

```
$ kubectl run -n demo --rm -i --restart=Never --image=curlimages/curl:8.11.1 t -- \
  sh -c 'for pw in admin456 admin123; do
    printf "admin/%s -> " $pw
    curl -s -o /dev/null -w "%{http_code}\n" -u admin:$pw http://<pod-ip>:8080/v
1/me
    done'
admin/admin456 -> 401
admin/admin123 -> 200
```

Nein. Miniflux hat den Admin beim allerersten Start mit admin123 in seiner Datenbank angelegt und liest ADMIN_PASSWORD seither nicht mehr („Skipping admin user creation because it already exists“). Das Passwort in OpenBao, im Kubernetes-Secret und im Container ist admin456; das Passwort, das zählt, liegt in der Miniflux-Datenbank und ist admin123.

Rotation im Secrets-Store ist nicht Rotation in der Anwendung. Für jeden Wert muss man wissen, ob die Anwendung ihn bei jedem Start übernimmt (Datenbank-URLs, API-Keys für Drittdienste: ja) oder nur beim ersten Mal (Bootstrap-Passwörter, Initial-Admins: nein). Bei Miniflux geht die Rotation über `miniflux -reset-password` – oder, konsequenter, indem man das Admin-Passwort gar nicht mehr über die Umgebung setzt, sondern nur einmal beim Bootstrap.

Das ist der Grund, warum dynamische Credentials (Nº 2) so viel wert sind: Da gibt es keinen Zustand in der Anwendung, der veralten kann.

Reloader: Restart bei Secret-Änderung

Für Fehler 3 gibt es eine Standardlösung. Stakater Reloader beobachtet Secrets und ConfigMaps und rollt jedes Deployment neu aus, das sie referenziert:

```
helm repo add stakater https://stakater.github.io/stakater-charts
helm upgrade --install reloader stakater/reloader --version 2.2.17 \
  --namespace reloader --create-namespace \
  --set reloader.watchGlobally=true
```

Eine Annotation am Deployment genügt:

```
metadata:
  name: miniflux
  namespace: demo
  annotations:
    reloader.stakater.com/auto: "true"
```

Dann die nächste Rotation – hier zurück auf admin123, damit Secret und Anwendung wieder übereinstimmen:

```
/ $ bao kv patch secret/demo/miniflux admin_password=admin123
version          4
$ kubectl annotate externalsecret -n demo miniflux-admin force-sync="$(date +%s)" --overw
ite

$ kubectl logs -n reloader deploy/reloader-reloader | grep miniflux
level=info msg="Changes detected in 'miniflux-admin' of type 'SECRET' in
namespace 'demo'; updated 'miniflux' of type 'Deployment' in namespace 'demo'"

$ kubectl get pod -n demo -l app=miniflux -o custom-columns=NAME:.metadata.name,CREATED:.m
etadata.creationTimestamp
NAME          CREATED
miniflux-7bf56c686c-lfss7  2026-09-17T10:34:17Z

$ kubectl exec -n demo deploy/miniflux -- sh -c 'echo $ADMIN_PASSWORD'
admin123
```

Wie Reloader das macht, sieht man im Deployment:

```
$ kubectl get deploy -n demo miniflux -o jsonpath='{.spec.template.spec.containers[0].env
[*].name}'
... STAKATER_MINIFLUX_ADMIN_SECRET
```

Es hängt eine Umgebungsvariable mit dem Hash des Secrets an den Container. Ändert sich der Hash, ändert sich das Pod-Template, und Kubernetes rollt aus – ein gewöhnlicher Rolling Update, mit allem, was dazugehört (Readiness, maxUnavailable).

Nebeneffekt, der die TTL-Falle aus N° 2 schließt: auto: "true" gilt für **alle** referenzierten Secrets, also auch miniflux-db. Alle 30 Minuten holt ESO neue Datenbank-Credentials, Reloader startet Miniflux neu, und der Pod läuft nie mit einem abgelaufenen User. Für einen RSS-Reader ist ein Neustart alle 30 Minuten in Ordnung. Für eine Anwendung, bei der er es nicht ist, muss die TTL länger sein oder das Secret als Datei gemountet und von der Anwendung neu gelesen werden.

Teil IV – Die anderen Lesarten

data mit property holt einzelne Felder. Drei weitere Formen, alle am Cluster getestet.

dataFrom.extract: das ganze Secret

```
spec:
  target:
    name: demo-app-extract
  dataFrom:
    - extract:
        key: demo/demo-app
```

Jedes Feld des KV-Secrets wird ein Key im Kubernetes-Secret, ohne Aufzählung:

```
$ kubectl get secret -n demo demo-app-extract -o jsonpath='{.data}'
{"password":"...","username":"..."}
```

Gleiche Policy wie bei data – read auf secret/data/demo/demo-app.

dataFrom.find: viele Secrets per Muster

```
spec:
  target:
    name: demo-find
  dataFrom:
    - find:
        path: demo
        name:
          regexp: ".*"
```

Beim ersten Versuch:

```
error processing spec.dataFrom[0].find, err: error getting all secrets:
cannot read secret data from Vault: Error making API request.

URL: GET http://openbao.openbao.svc:8200/v1/secret/metadata/demo?list=true
Code: 403.
```

find muss erst wissen, *welche* Secrets es gibt, und dafür listet es den Metadata-Pfad. Das ist eine andere Capability als read:

```
path "secret/metadata/demo"      { capabilities = ["list"] }
```

Und eine, die man bewusst vergibt: Wer list hat, sieht die Namen aller Secrets unter dem Präfix – auch derer, die er nicht lesen darf. Danach:

```
$ kubectl get secret -n demo demo-find -o jsonpath='{.data}'
{"demo_demo-app":"...", "demo_miniflux":"..."}

$ kubectl get secret -n demo demo-find -o jsonpath='{.data.demo_miniflux}' | base64 -d
{"admin_password":"...", "admin_username":"..."}
```

Pro gefundenem Secret ein Key (Pfad mit _ statt /), der Wert ist das ganze Secret als JSON.
 Brauchbar für Anwendungen, die ihre Konfiguration selbst parsen; für env-Variablen ungeeignet.

template: Dateien und zusammengesetzte Werte

```
spec:
  target:
    name: demo-app-template
    template:
      engineVersion: v2
      data:
        app.env: |
          APP_USERNAME={{ .username }}
          APP_PASSWORD={{ .password }}
        basic-auth: "{{ .username }}:{{ .password | b64enc }}"
      dataFrom:
        - extract:
            key: demo/demo-app
```

```
$ kubectl get secret -n demo demo-app-template -o jsonpath='{.data.app\.env}' | base64 -d
APP_USERNAME=...
APP_PASSWORD=...
```

Die Felder aus data/dataFrom sind im Template als .name verfügbar, dazu die Sprig-Funktionen (b64enc, upper, toJson, ...). So entsteht aus zwei Feldern eine .env-Datei, eine Verbindungs-URL (Nº 2 nutzt genau das für DATABASE_URL) oder eine httpasswd-Zeile.

Lebenszyklus: creationPolicy und deletionPolicy

Feld	Werte	Bedeutung
creationPolicy	Owner (Default), Orphan, Merge, None	Owner: ESO legt das Secret an und besitzt es. Merge: ESO schreibt nur seine Keys in ein bestehendes Secret. None: ESO legt nichts an
deletionPolicy	Retain (Default), Delete, Merge	Was passiert, wenn der Wert in OpenBao verschwindet: Retain behält das Secret, Delete löscht es

Was beim Löschen des **ExternalSecret** passiert, hängt von creationPolicy ab, nicht von deletionPolicy:

```
$ kubectl delete externalsecret -n demo demo-app-extract
$ kubectl get secret -n demo demo-app-extract
Error from server (NotFound): secrets "demo-app-extract" not found
```

Owner → OwnerReference → Garbage Collection. Wer das Secret nach dem Löschen des ExternalSecret behalten will, nimmt Orphan.

Teil V – PushSecret: die Gegenrichtung

Manche Secrets entstehen im Cluster: ein Token, das ein Operator erzeugt, ein Zertifikat von cert-manager, ein Kubeconfig eines Provisioners. Sie in OpenBao zu schreiben hat zwei Vorteile: Sie sind in den Raft-Snapshots gesichert, und jeder Lesezugriff steht im Audit-Log.

```
apiVersion: external-secrets.io/v1alpha1
kind: PushSecret
metadata:
  name: generated-token
  namespace: demo
spec:
  refreshInterval: 1h
  secretStoreRefs:
    - kind: SecretStore
      name: openbao
  selector:
    secret:
      name: generated-token
  data:
    - match:
        secretKey: token
        remoteRef:
          remoteKey: demo/generated-token
          property: token
```

Erster Versuch, mit einer Policy, die nur create/update auf secret/data/demo/generated-token erlaubt:

```
URL: GET http://openbao.openbao.svc:8200/v1/secret/data/demo/generated-token
Code: 403.
```

PushSecret **liest zuerst**, um zu vergleichen, ob sich etwas geändert hat. Also read dazu. Zweiter Versuch:

```
URL: PUT http://openbao.openbao.svc:8200/v1/secret/metadata/demo/generated-token
Code: 403.
```

Nach dem Schreiben der Daten schreibt ESO custom_metadata – eine Markierung, dass dieses Secret von ESO verwaltet wird. Das ist ein Write auf den **Metadata**-Pfad. Die vollständige Policy für einen Push-Pfad:

```
path "secret/data/demo/generated-token"      { capabilities = ["create", "read", "update"]
}
path "secret/metadata/demo/generated-token" { capabilities = ["create", "read", "update"]
}
```

```
$ kubectl get pushsecret -n demo generated-token -o jsonpath='{.status.conditions[0].message}'
PushSecret synced successfully

/ $ bao kv metadata get secret/demo/generated-token | grep -A1 custom_metadata
custom_metadata      map[managed-by:external-secrets]
```

Drei Pfade und drei Fehler für einen einzigen Push – die Policy für PushSecret ist deutlich breiter als für ExternalSecret. Man sollte sie auf genau die Pfade begrenzen, die gepusht werden, und nie secret/data/* schreibbar machen, nur weil ein Push nicht auf Anhieb funktioniert.

Wohin damit im Betrieb

ESO empfiehlt, PushSecret und ExternalSecret nicht auf denselben Pfad zu richten – zwei Controller, die dieselbe Wahrheit verwalten, sind ein Schwingkreis. Push für das, was der Cluster erzeugt; External für das, was OpenBao verwaltet.

Teil VI – Was schiefging, und warum

Fehler	Meldung	Ursache	Lösung
1	GET .../secret/data/demo/miniflux 403	Policy kennt den Pfad nicht	read auf secret/data/<pfad>
2	cannot find secret data for key: "admin_username"	kv put hat das Feld gelöscht	kv patch; Version 1 hat den Wert noch
3	Secret neu, Container hat alten Wert	env wird nur beim Start gelesen	Reloader, oder Secret als Datei mounten
4	Login mit neuem Passwort → 401	Anwendung liest den Wert nur beim Bootstrap	Rotation in der Anwendung, oder dynamische Credentials
find	GET .../secret/metadata/demo?list=true 403	find listet Metadata	list auf secret/metadata/<präfix>
Push	GET .../data/... 403, dann PUT .../metadata/... 403	Push liest, schreibt Daten, schreibt Metadata	create,read,update auf beide Pfade

Und das Muster dahinter: **READY False heißt nicht kaputt**. ESO lässt das Kubernetes-Secret unangetastet, solange der Sync scheitert. Die Anwendung läuft weiter, mit dem letzten guten Stand. Die Ursache steht nicht im ExternalSecret, sondern im Log des Operators:

```
kubectl logs -n external-secrets deploy/external-secrets | grep <name>
```

Teil VII – Betrieb

Die Policy, wie sie am Ende aussieht

```
path "secret/data/demo/demo-app"      { capabilities = ["read"] }
path "secret/metadata/demo/demo-app"  { capabilities = ["read"] }
path "secret/data/demo/miniflux"      { capabilities = ["read"] }
path "secret/metadata/demo"           { capabilities = ["list"] }
path "secret/data/demo/generated-token" { capabilities = ["create", "read", "update"]
}
path "secret/metadata/demo/generated-token" { capabilities = ["create", "read", "update"]
}
path "database/creds/demo-app"        { capabilities = ["read"] }
path "database/creds/miniflux"       { capabilities = ["read"] }
```

Acht Zeilen, jede wegen eines konkreten Fehlers hinzugefügt. Das ist der richtige Weg, eine Policy zu bauen – nicht `secret/*` mit allen Capabilities, sondern Pfad für Pfad, mit dem Fehler als Beleg.

Versionen als Sicherheitsnetz

```
/ $ bao kv get -version=1 secret/demo/miniflux
```

Nach einem misslungenen `kv put` ist der alte Stand eine Version zurück. `bao kv rollback -version=1 secret/demo/miniflux` macht ihn zur aktuellen Version (als neue Version 5, die Historie bleibt). ESO holt sie beim nächsten Sync.

refreshInterval wählen

Statische Secrets ändern sich selten; 1h ist ein guter Default. Kürzer bringt nichts außer Last auf OpenBao und Einträge im Audit-Log. Für den Moment nach einer Rotation gibt es die Annotation. `0` schaltet den periodischen Sync ab – dann syncet ESO nur bei Änderungen am `ExternalSecret`.

Was in OpenTofu gehört

Die Policy, die Kubernetes-Auth-Rolle, der KV-Mount. **Nicht** die Secret-Werte – die entstehen im Passwort-Manager und werden per `bao kv put` eingetragen, oder sie werden per `PushSecret` aus dem Cluster geschrieben. Ein `vault_kv_secret_v2` mit dem Passwort im HCL wäre genau der Fehler, den ESO vermeiden soll.

Was noch offen ist

- Das Miniflux-Admin-Passwort wird bei jedem Start in die Umgebung gesetzt, aber nur beim ersten benutzt. Sauberer: `CREATE_ADMIN` nur für den Bootstrap, danach aus dem Manifest entfernen.

- Der ESO-Operator selbst hat clusterweite Rechte auf Secrets. Wer ihn kompromittiert, liest alle. Das ist der Preis des Musters, und er ist der Grund, warum die OpenBao-Policy pro Namespace so eng wie möglich sein sollte.

Anhang A – Alle Kommandos

```
# — KV —————
bao kv put    secret/demo/miniflux admin_username=admin admin_password=admin123
bao kv patch secret/demo/miniflux admin_password=admin456      # ein Feld ändern
bao kv get    -version=1 secret/demo/miniflux                  # alte Version lesen
bao kv rollback -version=1 secret/demo/miniflux                # zurück
bao kv metadata get secret/demo/miniflux

# — Policy (root oder admin) —————
bao policy write eso-demo - < k8s/eso-demo-policy.hcl

# — ESO —————
kubectl apply -f k8s/secretstore.yaml
kubectl apply -f k8s/miniflux-admin-external-secret.yaml
kubectl apply -f k8s/demo-app-extract.yaml -f k8s/demo-find.yaml -f k8s/demo-app-template.
yaml
kubectl apply -f k8s/push-secret.yaml
kubectl get externalsecret,pushsecret -n demo
kubectl annotate externalsecret -n demo <name> force-sync="$(date +%s)" --overwrite
kubectl logs -n external-secrets deploy/external-secrets | grep <name>

# — Reloader —————
helm upgrade --install reloader stakater/reloader --version 2.2.17 \
  -n reloader --create-namespace --set reloader.watchGlobally=true
kubectl annotate deploy -n demo miniflux reloader.stakater.com/auto="true"
kubectl logs -n reloader deploy/reloader-reloader | grep <secret>
```

Anhang B – Glossar

KV v2 – Versioniertes Key-Value-Engine. Pfade `data/` (Inhalt) und `metadata/` (Versionen, Zeitstempel, `custom_metadata`).

SecretStore / ClusterSecretStore – ESO-Objekt, das Provider, Adresse und Auth beschreibt. Namespace-gebunden bzw. clusterweit.

ExternalSecret – Auftrag an ESO, Werte aus dem Store in ein Kubernetes-Secret zu schreiben. `data` (Felder), `dataFrom.extract` (ganzes Secret), `dataFrom.find` (Muster).

PushSecret – Gegenrichtung: Kubernetes-Secret → Store.

remoteRef.key – Pfad im Store ohne Mount und ohne `data/`; ESO ergänzt beides.

property – Ein Feld innerhalb eines KV-Secrets.

template – Go-Template mit Sprig-Funktionen, das aus den geholten Feldern die Keys des Ziel-Secrets baut.

creationPolicy – Wem das Ziel-Secret gehört (Owner, Orphan, Merge, None).

deletionPolicy – Was passiert, wenn der Wert im Store verschwindet.

refreshInterval – Wie oft ESO den Store abfragt. Eine Annotation-Änderung erzwingt einen Sync sofort.

Reloader – Controller, der Deployments neu ausrollt, wenn referenzierte Secrets oder ConfigMaps sich ändern; hängt dazu einen Hash als Env-Variable an.

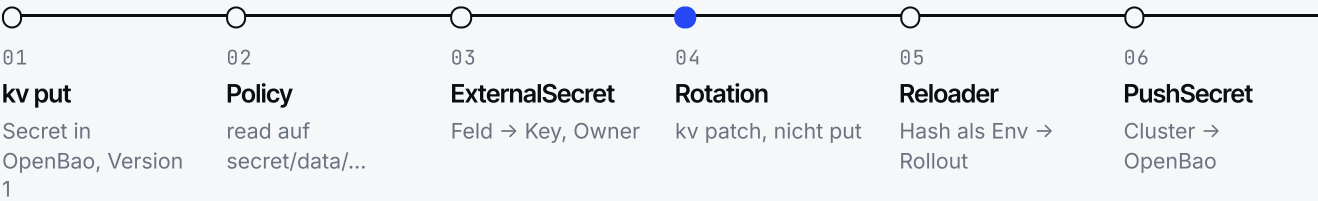
kv put vs. kv patch – `put` schreibt eine neue Version mit genau den angegebenen Feldern; `patch` ändert nur die angegebenen.

Das Secret sagt admin456. Die Anwendung akzeptiert admin123. Beide haben recht.

Dieser Leitfaden holt statische Secrets mit dem External Secrets Operator aus OpenBao in den Cluster – Feld für Feld, als ganzes Secret, per Muster, als gerenderte Datei – und schreibt sie per PushSecret zurück. Jede Variante am Cluster getestet, mit der Policy-Zeile, die sie braucht.

Und vier Fehler, die jede Rotation lauern lässt: die Policy, die den Pfad nicht kennt; kv put, das ein Feld löscht; der Pod, der das neue Secret nie liest; die Anwendung, die das Passwort nur beim ersten Start übernimmt. Reloader löst einen davon.

```
-- Der Unterschied, der ein Feld gelöscht hat
bao kv put    secret/demo/miniflux admin_password=... # ersetzt alles
bao kv patch  secret/demo/miniflux admin_password=... # ändert eines
```



openbao	2.6.2 · kv v2 · secret/
external-secrets	2.10.0 · v1 · PushSecret v1alpha1
reloader	chart 2.2.17 · watchGlobally
miniflux	2.3.3
policy	8 Zeilen · 6 Fehler

26 Seiten · 7 Teile · 2 Anhänge
Vertiefung: **Vault in Practice**
leanpub.com/vault-in-practice
Gesetzt in Inter Tight, Inter und JetBrains Mono.
Alle Ausgaben sind echte Terminal-Ausgaben.
CC BY 4.0 · Weitergabe erlaubt · ohne Gewähr